

Internal Document RAG Chatbot - R&D Documentation

Project Start Date: 27 May 2026

Target Completion Date: 09 July 2026

Working Schedule: Monday to Friday, 2:00 PM – 6:00 PM (4 hours per working day)

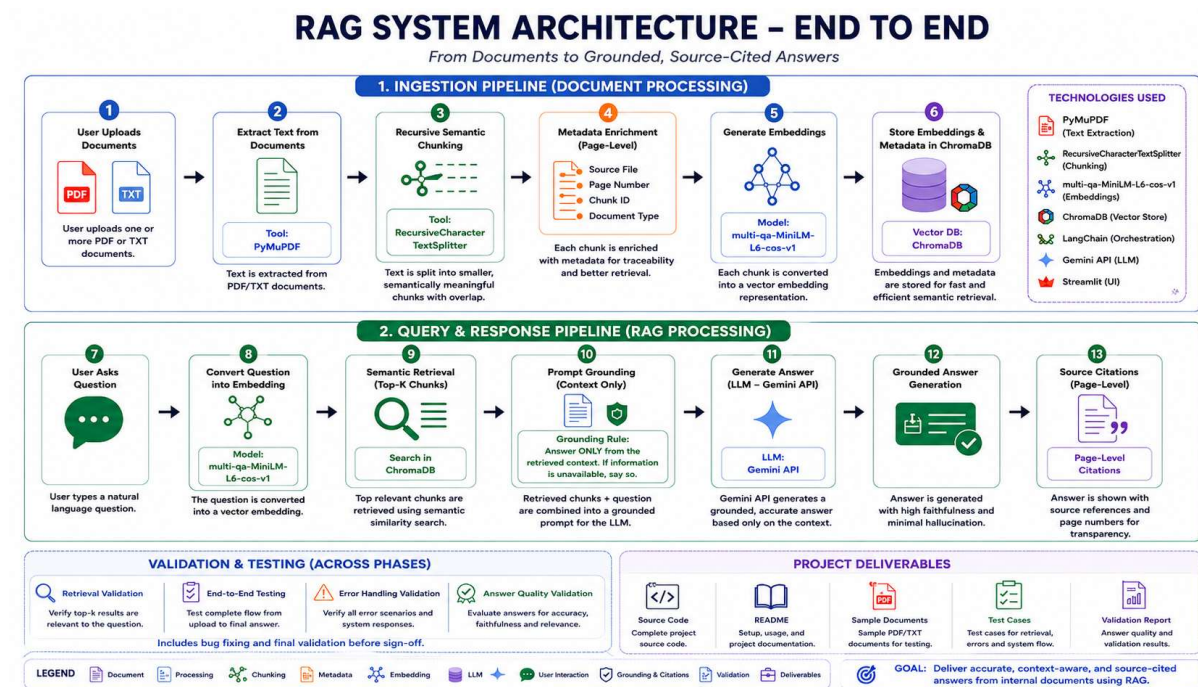
1. Summary

The Internal Document RAG Chatbot is an AI-powered document search and question-answering system designed to help users retrieve information from uploaded PDF and TXT documents. The system combines semantic search, document metadata, and grounded answer generation to provide source-backed responses with page-level citations.

2. Project Scope

- Upload and process PDF/TXT documents
- Extract and preprocess document text with page tracking
- Apply recursive semantic-preserving chunking with overlap
- Enrich each chunk with metadata (source_file, page_number, chunk_id, document_type)
- Generate embeddings for semantic retrieval
- Store embeddings and metadata in ChromaDB vector database
- Accept user questions through Streamlit UI
- Retrieve top relevant chunks using vector similarity search
- Apply strict prompt grounding to reduce hallucination (context-only answering)
- Generate grounded answers using Gemini API
- Display answer with page-level source citations

3. Architecture Diagram



4. Technology Stack

Component	Technology
Frontend	Streamlit
Backend	Python (Streamlit handles backend)
PDF Extraction	PyMuPDF
Chunking	RecursiveCharacterTextSplitter
Metadata Layer	ChromaDB metadata fields
Embeddings	multi-qa-MiniLM-L6-cos-v1
Vector Database	ChromaDB
Prompt Grounding	Custom prompt template
LLM	Gemini API
Framework	LangChain

Note: FastAPI has been excluded from the initial implementation as Streamlit and Python modules are sufficient for the current project scope. FastAPI can be introduced in future if external API exposure or multi-client integrations are required.

5. Future Enhancements

- Conversation memory for contextual multi-turn chat sessions
- Structured Table Extraction using pdfplumber / PyMuPDF / Camelot
- Multimodal RAG support for images, diagrams, and captions

6. Effort & Timeline Estimation

Step 1: R&D, Architecture Planning & Requirement Analysis

Scope

- * Understand RAG architecture and workflow
- * Analyze embeddings, vector databases, and retrieval pipelines
- * Finalize technology stack
- * Design metadata schema
- * Prepare architecture and SDLC documentation

Timeline : 27 May 2026 – 05 June 2026 (9 Working Days)

Task Breakdown

RAG architecture research – 27 May 2026

Research paper analysis – 28 May 2026

Technology stack finalization – 29 May 2026

Metadata schema planning – 01 June 2026

SDLC planning and documentation – 02 June 2026

Architecture review and validation – 03-05 June 2026

Deliverables

- * Finalized architecture
- * Technology stack selection
- * Metadata schema design
- * R&D documentation

Completion Target

05 June 2026

Step 2: Environment Setup & Project Initialization

Scope

- * Setup development environment
- * Configure Streamlit
- * Configure Gemini API
- * Create project structure

Timeline : 08 June 2026 – 09 June 2026 (2 Working Days)

Task Breakdown

Python virtual environment setup – 08 June 2026

Dependency installation – 08 June 2026

Streamlit project setup – 09 June 2026

Gemini API configuration – 09 June 2026

Environment & API integration validation testing – 09 June 2026

Deliverables

- * Working development environment
- * Configured project structure
- * API setup completed

Completion Target

09 June 2026

Step 3: Document Upload & Text Extraction Pipeline

Scope

- * Implement PDF/TXT upload functionality
- * Integrate PyMuPDF extraction
- * Extract page-level content
- * Validate extracted text

Timeline : 10 June 2026 – 12 June 2026 (3 Working Days)

Task Breakdown

File upload module development – 10 June 2026

PDF extraction using PyMuPDF – 10 June 2026

TXT file handling – 11 June 2026

Page tracking integration – 12 June 2026

Extraction validation and testing – 12 June 2026

Deliverables

- * Working upload pipeline
- * Page-aware text extraction system

Completion Target

12 June 2026

Step 4: Chunking & Metadata Enrichment Pipeline

Scope

- * Implement recursive semantic-preserving chunking
- * Configure chunk overlap
- * Generate metadata
- * Validate chunk quality

Timeline : 15 June 2026 – 17 June 2026 (3 Working Days)

Task Breakdown

RecursiveCharacterTextSplitter implementation – 15 June 2026

Chunk size and overlap tuning – 16 June 2026

Metadata integration (source, page, chunk id) – 17 June 2026

Chunk validation and semantic continuity testing – 17 June 2026

Deliverables

- * Recursive chunking pipeline
- * Metadata-aware chunk storage

Completion Target

17 June 2026

Step 5: Embedding Generation & Vector Database Integration

Scope

- * Generate embeddings
- * Store vectors in ChromaDB
- * Persist embeddings with metadata

Timeline : 18 June 2026 – 22 June 2026 (3 Working Days)

Task Breakdown

multi-qa-MiniLM-L6-cos-v1 integration – 18 June 2026

Embedding generation pipeline – 19 June 2026

ChromaDB integration – 22 June 2026

Metadata persistence validation – 22 June 2026

Embedding quality validation – 22 June 2026

Deliverables

- * Embedding pipeline
- * ChromaDB integration
- * Persistent vector storage

Completion Target

22 June 2026

Step 6: Retrieval Pipeline & Semantic Search

Scope

- * Generate question embeddings
- * Retrieve top-k relevant chunks
- * Implement semantic search

Timeline : 23 June 2026 – 25 June 2026 (3 Working Days)

Task Breakdown

Question embedding generation – 23 June 2026

Similarity search implementation – 23 June 2026

Top-k retrieval implementation – 24 June 2026

Retrieval validation – 25 June 2026

Retrieval accuracy testing – 25 June 2026

Deliverables

- * Semantic retrieval system
- * Metadata-aware search pipeline

Completion Target

25 June 2026

Step 7: Prompt Grounding & Gemini API Integration

Scope

- * Create grounded prompts
- * Integrate Gemini API
- * Generate context-grounded answers

Timeline : 26 June 2026 – 29 June 2026 (2 Working Days)

Task Breakdown

Prompt template creation – 26 June 2026
Grounding rule implementation – 26 June 2026
Gemini API integration – 29 June 2026
Response validation – 29 June 2026
Grounded response validation – 29 June 2026

Deliverables

- * Grounded prompt pipeline
- * Gemini-powered answer generation

Completion Target
29 June 2026

Step 8: Citation Rendering & Streamlit UI

Scope

- * Build chatbot interface
- * Display source citations
- * Display retrieved source chunks

Timeline : 30 June 2026 – 06 July 2026 (3 Working Days)

Task Breakdown

Chat interface implementation – 30 June 2026
Upload panel integration – 30 June 2026
Source citation rendering – 01 July 2026
Source chunk display – 01 July 2026
UI testing and improvements – 06 July 2026

Deliverables

- * Functional chatbot UI
- * Citation-aware frontend

Completion Target
06 July 2026

Step 9: Testing, Debugging & Final Optimization

Scope

- * Validate complete workflow
- * Fix issues
- * Optimize performance

Timeline : 07 July 2026 – 09 July 2026 (3 Working Days)

Task Breakdown

End-to-end workflow testing – 07 July 2026
Retrieval accuracy validation – 07 July 2026
Hallucination guard testing – 08 July 2026
Bug fixing and cleanup – 08 July 2026
Answer quality validation report preparation – 09 July 2026
Final project validation – 09 July 2026

Deliverables

- * Stable working prototype
- * Optimized RAG workflow
- * Final validated system
- * Answer quality validation report

Completion Target
09 July 2026

7. UI Layout

The application uses a three-zone Streamlit layout, mapped directly to the project scope.

- **Left sidebar:** Document uploader; list of ingested documents with per-file chunk counts; total chunks metric; settings (top-k chunks, LLM temperature); GOOGLE_API_KEY status.
- **Center (chat window):** Past chat turns rendered as user / assistant message bubbles. Each turn carries a collapsed inline expander listing the sources used for that specific answer.
- **Bottom (source references panel):** Persistent panel showing the latest answer's retrieved chunks as cards laid out two per row. Each card displays the marker [#N],

source_file, page_number, distance score, and a 'Show chunk text' expander that reveals the full chunk and chunk_id.

- **Chat input:** Streamlit chat_input docked at the bottom of the page for submitting new questions.

Mockup:

+-----+-----+	
Documents	Chat Window
PDF 1	User Question
PDF 2	
PDF 3	AI Answer with [#N] citations
+-----+-----+	
Source References (last answer)	
+-----+-----+	

8. Error Handling

Each error case raises a typed exception in the document loader and is surfaced to the user with a clear UI message. The system behavior (skip / reject / no LLM call) is fixed per case; users may re-submit manually but there is no automatic retry.

- **Corrupted PDF**

Behavior: File skipped; ingestion continues with the remaining files.

Message shown: "<filename> appears corrupted — skipped"

- **Encrypted PDF**

Behavior: File skipped; ingestion continues with the remaining files.

Message shown: "<filename> is password-protected — skipped"

- **Unsupported file type**

Behavior: Upload rejected before the file is saved to disk.

Message shown: "Unsupported file type: <ext>. Supported: ['.pdf', '.txt']"

- **File too large**

Behavior: Upload rejected before the file is saved to disk (limit 50 MB).

Message shown: "<filename> is <size> MB — limit is 50 MB"

- **Empty PDF / no extractable text**

Behavior: File skipped with a warning (OCR is out of scope for this version).

Message shown: "No extractable text found in <filename>. This may be a scanned / image-only PDF."

- **Empty TXT**

Behavior: File skipped with a warning.

Message shown: "<filename> is empty"

- **Duplicate upload**

Behavior: Duplicate uploads are detected using a SHA-256 file hash generated from the document content. If the same document is uploaded again, ingestion is skipped and existing records are reused.

Message shown: "This document has already been uploaded. Existing data will be reused."

- **Gemini API failure**

Behavior: Error message shown inside the assistant message bubble; the question is not retried automatically.

Message shown: "Gemini call failed: <error>"

- **Missing GOOGLE_API_KEY**

Behavior: Sidebar banner shown; no LLM call is made until the key is set.

Message shown: "GOOGLE_API_KEY not set — add it to .env to enable answers"

- **No relevant answer found**

Behavior: Before sending the retrieved context to Gemini, the system performs a retrieval relevance check based on similarity scores. If no retrieved chunks meet the minimum relevance threshold, the Gemini API is not called.

Message shown: "The document does not contain sufficient information."

- **Empty vector store (question asked before any ingestion)**

Behavior: No LLM call is made; the user is asked to ingest documents first.

Message shown: "Ingest some documents first — the vector DB is empty"

9. Answer Quality Validation

Answers generated by the chatbot are validated by manual comparison against the source documents, using the page-level citations the system surfaces. Validation is performed during Step 9 and during stakeholder review; it is also expected as a continuous check by every reviewer.

- **Citation traceability:** Every grounded answer carries [#N] markers tied to the cards in the bottom Source References panel. For each cited chunk, the reviewer opens the named source_file at the listed page_number and confirms the cited text is present and supports the answer.
- **Fact-level spot checks:** A predefined evaluation set is used (single-doc recall, cross-document, numeric / date precision, tricky overlapping concepts). For each question the reviewer records: correct answer text Y/N, correct citation source Y/N, correct page number Y/N.
- **Hallucination guard checks:** Questions whose answer is not in any ingested document are asked deliberately. The system must reply exactly 'The document does not contain sufficient information.' - any fabricated content is logged as a defect.
- **Retrieval quality:** The reviewer inspects the distance scores in the source cards. A retrieved chunk with a distance score noticeably higher than its peers is flagged as a likely irrelevant hit and reviewed in context.
- **Edge case verification:** Corrupted PDF, oversized upload, empty PDF, duplicate upload, missing GOOGLE_API_KEY, and Gemini API failure are exercised explicitly and the resulting UI message is captured.
- **Pass criteria:** A test run passes when (a) all factual-recall questions are answered correctly with the right source and page, (b) all hallucination-guard questions are refused with the exact refusal phrase, and (c) every error-handling case produces its expected UI message.

Evaluation Question Set

A small evaluation set executed during Step 9 testing. The tester fills in 'Actual Answer' and 'Pass/Fail' per row and the completed table becomes the Answer Quality Validation Report deliverable.

#	Question	Expected Answer	Expected Source	Expected Page	Actual Answer	Pass / Fail	Remarks
1	How many PTO days do I get	22 working days	employee_handbook.txt	1			

	per year?			
2	What is the minimum password length?	14 characters	security_policy.txt	1
3	When is Atlas 3.5 releasing?	15 July 2026	product_atlas_specs.txt, q2_planning_meeting_notes.txt	1
4	Who chaired the Q2 planning meeting?	Priya Raman	q2_planning_meeting_notes.txt	1
5	On what date was Atlas 3.4 released?	28 February 2026	product_atlas_specs.txt	1
6	Can I work fully remote if I live near the office?	No - requires home base more than 80 km from the nearest office	employee_handbook.txt	1
7	Who is the CEO of Nexotech ?	"The document does not contain sufficient information."	(hallucination guard - not in any document)	—
8	What is the WiFi password in the Bangalore office?	"The document does not contain sufficient information."	(hallucination guard - not in any document)	—

10. Final Deliverables

End-of-project artefacts handed over together as the project package:

- **Source code:** Complete Python project under rag_chatbot_project/ (app modules, Streamlit entry main.py, configuration).
- **README.md:** Setup, run, and test instructions, including the required .env GOOGLE_API_KEY entry.
- **Sample documents:** Four representative internal documents under sample_docs/ for exercising ingestion and retrieval (employee_handbook.txt, product_atlas_specs.txt, security_policy.txt, q2_planning_meeting_notes.txt).
- **Validation Sheet:** Manual validation results covering retrieval accuracy, citations, hallucination checks, and error handling scenarios.
- **Answer quality validation report:** The Evaluation Question Set table in section 10, completed with actual answers and pass/fail per row, taken during Step 9 testing.
- **R&D documentation:** This document - architecture, scope, plan, error handling, UI layout, validation process, and deliverables list.